

Prolog Minimax Assignment Writeup

Isabella Read and Luis Ruiz

1. Why is minimax optimal under the assumptions (deterministic, perfect information, zero-sum)?

The minimax algorithm is optimal for deterministic, perfect information, zero-sum games because it allows players to map all possible outcomes based on a game's state that prioritizes their most optimal actions while limiting the opponent's most optimal response. Variation from non-deterministic, non-zero-sum games would add increased difficulty towards mapping potential game states, rendering the algorithm less effective.

2. What does a minimax value mean “in words”?

A minimax value tells how good or bad a position will end up if everyone plays the best possible moves, or perfectly. It's the outcome that can be guaranteed from a position if both players make the best possible moves.

3. Why is minimax infeasible for large games like chess?

Minimax is infeasible for chess because the game tree is enormous. There are a huge number of possible moves and positions in chess, so trying to explore every possible future move would involve an unrealistic amount of time and computational power.

Node Expansion Counts and Comparison

Board	Player	Value	Node Expansions
[x,o,e,e,x,e,o,e,e]	x	1	17273
[x,o,x,x,o,o,o,x,e]	x	0	607
[x,o,x,e,o,e,e,e,x]	o	-1	605

Code Comparison to ChatGPT

Move/3: Our implementation of move/3 used member/2 and append/3. Member/2 will pick an empty cell, append/3 will split the board around that spot, and then a new board will be created with append/3 and adding the players character in between the front and back of the board. ChatGPT used nth1/3 and nth1/4. Nth1/3 finds an empty position and then nth1/4 replaces that position with the player. It seems like ChatGPT used something more advanced than we did and it may also be faster.

Terminal/1: Our implementation compared to ChatGPT is the same. Our assumption is because terminal/1 is such a simple function to write and there are other helper functions, like win and full, that help to make the function even more simple. The semicolons or the “OR” statements for the function and if one of those statements don’t return true, then terminal/1 will return false.

Utility/2: ChatGPT provided an implementation of utility formatted similarly to our implementation with slight improvements in efficiency. Rather than setting U to (1,0,-1) based on the win condition, the generated implementation returns the desired value based on the winner inside the predicate head. The predicate for a tie also checks for the board to be full before returning a value.

Best_move/4: Our implementation of best_move finds the potential moves based on the given game state, gets the minimax value of the successor state and recurses through the rest of the moves to find the optimal choice. ChatGPT generated a solution that maps the potential next moves. It then evaluates the min max of all the successor states using findall and uses a helper to select the optimal choice depending on if the player is x or o, rather than individually comparing the states. This implementation seems more advanced and likely more efficient.